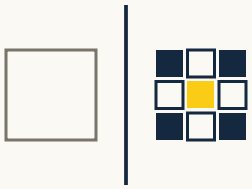


You Don't Buy a Transformation. Your People Build One.

Lens 1 of 3 — the people who run the work, and the result they learn to own

By Robert Paddock · idigdata

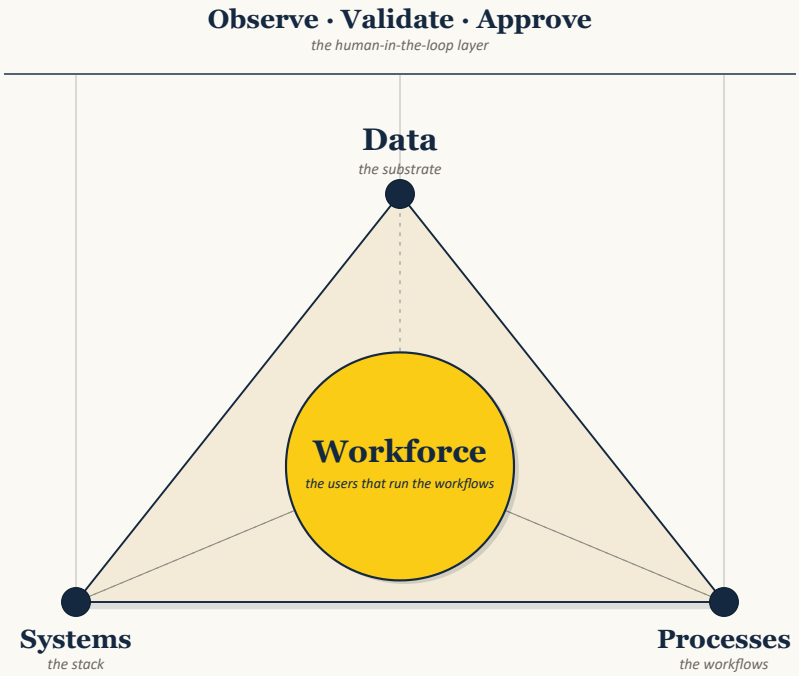
After thirty years and fifty-plus business-system transformations inside $100M–$1B operating companies, I have yet to see a company walk into the work with its people, data, workflows, systems, and decisions already mapped into one living operating model.

That is not an indictment. It is the work.

The people know more than the systems show. They know which spreadsheet is trusted even though nobody admits it. They know which field in the ERP means one thing to Finance and another thing to Operations. They know which customer record is safe, which approval path is real, which workaround keeps the floor moving, and which process nobody has documented because the person carrying it has always just handled it.

Transformation starts there.

Not with the platform pitch. Not with the steering deck. Not with the promise that a new system will force discipline into the business after go-live. Transformation starts with the people who run the work, the data they touch every day, and the workflows they keep alive when the official process breaks.



Data, systems, and processes — with the people who run the work at the center.

That is why most transformation programs struggle. Bain has put the failure gap bluntly: 88% of transformations fall short of their ambitions. In my experience, they rarely fall short because the company chose the wrong software. They fall short because the program never truly discovered the operating reality of the business, never made that reality visible, and never transferred ownership to the people who would have to run the result after the consultants left.

The work has to leave more than a configured platform. It has to leave governed data, mapped workflows, visible decisions, and operators who can run the result.

The people layer is not a workstream

Most programs treat the people side as a parallel lane. Technology over here. Change management over there. Training near the end. Communications when morale dips. Adoption after go-live.

That split is where the decay begins.

The people who run the work are not separate from the system. They are the system's center. If they cannot see the new model, test it, challenge it, improve it, and eventually own it, the transformation is theater with a go-live date.

Real change does not ask people to accept mystery. It asks them to help make the business visible.

The best operators in a company are not obstacles to transformation. They are the map. When they resist, it is often because they are being handed bad change: new process nobody explained, new data rules nobody governed, new automation nobody can challenge, new metrics that ignore the work as it actually happens. People are not usually resistant to change. They are resistant to being ignored while bad change is done to them.

A serious transformation respects that signal. Respecting the signal does not mean every local preference wins; leadership still has to decide what becomes enterprise standard, what stays local, and who owns the rule after the meeting ends.

It does not romanticize every workaround. Some workarounds are risk. Some tribal knowledge is fragility. Some local control has to become enterprise governance. But the way to get there is not to flatten the people layer. It is to surface it, structure it, and build with it.

I have the scar to prove it. On one program I had everything the textbook says you need. The technology was specced. The requirements were solid. The SMEs were pulling hard, and the executive team was behind it. What I did not have was the layer in between. The senior leaders who sit above the SMEs and below the executive team were never truly aligned, and new leaders were arriving while the work was underway. The SMEs designed the workflows to their understanding of the business. The senior leadership team had a different picture of what those workflows needed to do. The executive team sat a level too far removed to see the gap forming. By the time it surfaced, it cost six months to settle the vision, re-align every party, and rebuild confidence in the design. The lesson I never unlearned: engaging the top and the bottom is not enough. The middle has to be aligned to the same picture of the work, because that is the altitude where the design meets the intent, and it is exactly the altitude a transformation is most tempted to skip.

What client-owned really means

A client-owned transformation is not just a data architecture decision. It is an operating decision.

It means the business owns the data model, the process map, the decision record, the governance cadence, the SOPs, the registers, and the improvement loop. It means the people inside the business can maintain the system without calling the consultant every time the business changes. The work cannot live only in a partner's framework, a vendor's roadmap, or one heroic employee's desktop folder.

The familiar model has a different center of gravity. License the platform. Hire the partner. Integrate on the vendor's calendar. Let the customer master, item master, process logic, and reporting logic live inside someone else's product structure. Pay the implementation fees, the support contracts, the upgrade cycles, and the migration bills every time the vendor's world changes.

That model can produce a working system. It often does. But it does not necessarily produce a business-owned transformation.

Someone has to be accountable above the vendor lanes: holding the business model, the data model, the workflow decisions, and the ownership transfer together until the internal team can run it, then leave.

Happy systems, happy people

When the model lands, the company feels different from the inside.

People stop spending their days reconciling systems that should agree. Finance stops rebuilding the truth every month. Operations stops making decisions from stale reports. IT stops being the permanent translator between business reality and vendor constraints. The people closest to the work can see the process, name the exception, trace the decision, and improve the system without starting from zero.

That is what happy systems make possible.

The architecture matters because it gives the people a stable floor. A Common Data Model the client owns. Master data governed at the enterprise level. ERP, CRM, WMS, HRIS, FP&A, MES, and other applications treated as distinct execution layers of an ecosystem rather than the center of the operating model. Integrations governed by the client. Data that remains where the business put it, in a schema the business owns, under a cadence the business can maintain.

But the architecture only earns its keep when the people can use it.

The transformation produces a composite operating result: systems that fit the operating model, data that is audit-defensible, process documentation that stays current

because it is tied to the work, decision trails that survive personnel changes, and SOPs that remain alive after go-live.

The point is not to make the company dependent on a smarter consultant. The point is to leave the business stronger than it was when the engagement began.

Happy systems, happy people.

How we know we have arrived

Arrival is not a ceremony. It is a change in daily reality.

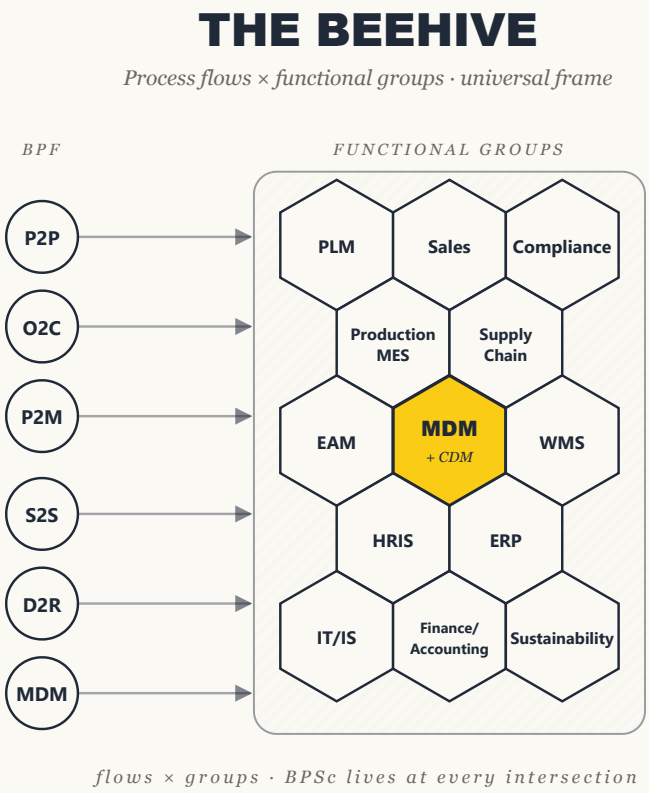
The CFO closes with fewer reconciliations because the chart of accounts and master data finally line up across the business. A customer record means the same thing across Sales, Finance, and Operations. A new acquisition plugs into a known data and workflow structure instead of triggering another eighteen-month systems event. An internal IT lead can make a bounded change without escalating to the implementation partner. A process owner can point to the decision record and explain why the workflow works the way it does.

Some markers are universal. Books close cleaner. Exceptions become visible earlier. Audit questions are easier to answer. Vendor changes stop feeling existential. Reporting becomes less theatrical and more operational.

Some markers are specific to the operator. A construction business may measure arrival through project-cost variance and joint-venture reporting. A distributor may measure it through customer-master quality and national-account pricing discipline. A manufacturer may measure it through product-line margin and standard-cost trust. The daily pain is different. The structural answer is the same: people, data, and workflows made visible enough to govern.

The transformation reaches the destination when the people who run the business can say, with evidence: this used to be fragile, now it is governed; this used to live in someone's head, now the business owns it; this used to require outside rescue, now our team can run it.

The beehive



Standard frame · bespoke fill · 100s of BPSc per BPF, unique per operator

The beehive — process flows by functional groups, a universal frame.

The people map is not the org chart.

The org chart shows reporting lines. The beehive shows how work actually moves: functional groups, cross-functional workflows, exception paths, decision rights, handoffs, and the informal authorities who know where the bodies are buried.

The beehive is the organizing model for the people layer. Leads and SMEs are grouped by functional area: Sales, Production, Supply Chain, Finance and Accounting, IT/IS, and the operator's other real functional groups. Around that grid sit the core business process constellations: Procure to Pay, Order to Cash, Plan to Manufacture, Systems to Support, Data to Reporting, and Master Data Management.

The frame repeats across $100M–$1B operating companies; the population, pressure points, and decision rights are always specific.

That distinction matters. A generic pod model often misses the cross-functional flows that run the business. Sales touches Finance. Operations touches Supply Chain. IT touches every workflow but does not own every decision. Master data crosses all of it.

When the AR and Sales standoff fires, it does not get solved by a slogan about alignment. It gets solved at the business process step where revenue recognition, customer setup, pricing, invoicing, and collection touch. When Operations and Finance disagree about inventory truth, it gets solved where physical movement, cost, and reporting meet. The beehive gives the company a way to locate the real argument and resolve it in the work.

People can trust a transformation faster when they recognize their operating truth inside it.

Tribal knowledge is not the enemy

Every transformation has its spreadsheet person.

The person may not have the title. They may not be in the steering committee. They may not be the executive anyone names first. But somewhere in the company, a person or small group is carrying the real logic of the business in spreadsheets, saved reports, private checklists, desktop files, and memory.

This is operational reality.

Sometimes that person is protecting the business from bad systems. Sometimes they are also creating risk because the company has no governed way to replace what they know. Both can be true.

A serious transformation does not shame that person or automate around them blindly. It brings them into the work early. It asks what the spreadsheet knows that the system does not. It asks who approves changes, who trusts the output, what breaks if the person is unavailable, and which parts of the logic should become governed enterprise process.

I once walked into a manufacturer where the entire cost structure lived in one person's spreadsheet. Not a tab. A web of linked Excel workbooks, genuinely powerful, that pulled machine times, route times, and labor times and set the standard product cost for everything the company made. Over years, that model had quietly become the source of truth. Every downstream department had calibrated to its formulas instead of to the actuals on the floor. Finance managed the variance against the sheet, so the sheet's math, not the real machine and labor times, had become the company's reality. The first useful conversation was not about a new system. It was sitting with the person who owned that model and walking the real rule: where each number came from, which times were measured and which were assumed, who could change a formula, who checked it, and what the business would lose if that file disappeared tomorrow. That is what gets written down in the first weeks, as decision logic, ownership, and risk,

not as a training note. The goal is not to rip the model out from under the person who built it. It is to make what they know governed and visible, so the business can finally monitor against actuals instead of against accounting magic.

This is where data governance becomes a people conversation before it becomes a technical one.

If the business cannot agree what a customer is, what an item is, what a project is, what a location is, what a margin calculation means, or who owns a master-data change, the problem will not be solved by upload. It started before the upload. The implementation did not fail at go-live. It failed at the governance conversation that never happened.

When the conversation finally happens, the people who carried the workaround can become part of the design authority instead of being treated as cleanup after the fact.

The system is alive because the company is

A transformation that depends on the consultant after go-live has not fully transferred.

The handoff moment should be concrete. The operator's internal team can log into the system with ownership rights. The data-governance cadence is active. The issue register is being triaged by the client-side team. Process owners know how to challenge and improve the workflow. Internal leads are already running the next improvement cycle. SOPs are not a folder of stale documents; they are produced and maintained through the operating fabric.

The system does not pause when the consultant leaves. It keeps running.

The engagement structure should force the question: what must the business own before the outside operator exits? The answer is not everything in the consultant's head. The answer is a working operating fabric the business can maintain: data,

workflows, decision records, governance roles, taskforces, SOPs, and improvement cadence.

Continuous improvement is not an afterthought. It is the sign that ownership has transferred. Stable state is not the end. It is the gate into a better operating rhythm, where every meaningful business change can be assessed, designed, tested, deployed, and absorbed through the same fabric.

The system is alive because the company is.

Why architecture still matters

The people lens does not make architecture optional. It makes architecture accountable.

A business cannot ask people to own a system that is structurally owned by someone else. If the customer master lives only inside the vendor's tables, if the core process logic lives only inside partner configuration, if the reporting layer is a pile of extracts, and if every meaningful change needs the vendor's hands, the people layer will eventually decay back into workarounds.

Data and integration ownership have to be structural because when they are not, even a small change becomes a dependency event.

That means a client-owned Common Data Model, master data governed at the enterprise level, applications bounded into their domains, vendor partners contributing what they do well, and decisions traced where Finance, Compliance, Operations, IT, and the people carrying the work can see them.

The details matter, but they belong in the delivery mechanics: which data belongs in the common model, which governance roles own it, which activities capitalize, which audit

trails are required, which vendor lanes stay bounded, and how the program is paid for without turning into a billable-hours dependency machine.

Those mechanics are the subject of Article 2, the mechanics. The point here is simpler: architecture is only successful when the operating company can own it.

Who this is for

This model is for mid-market operators who want the business to own the result.

Not every company wants that. Some organizations prefer vendor-managed risk. Some want a platform to absorb responsibility. Some want the comfort of a large partner and a familiar implementation playbook. That is a valid choice.

This model asks more of the client. It asks executives to own decisions instead of delegating every hard call to the vendor. It asks internal SMEs to step into the work while they are already carrying full-time jobs, which means the work has to be bounded, scheduled, and protected rather than dumped on top of fifty-hour weeks. It asks the business to govern data as an operating discipline, not as a cleanup task before upload.

For the companies that are ready, the work is worth it.

The people get systems that respect the work instead of fighting it. The business gets governed data, mapped workflows, visible decisions, and an operating model it can extend. Vendors stay useful inside their lanes. The internal team grows stronger through the build. The consultant exits by design.

Article 2, the mechanics, covers how the engagement gets delivered, governed, funded, and defended. Article 3, Production Agentics: The Business Asset, shows how this same people-readiness gate moves one substrate higher.

The test is simple. When I leave, the business can still see the work, govern the data, improve the workflows, and run the next change without starting over. The business keeps the asset. That was always the point.

Bring the real situation. If this is the transformation you are trying to run, start a conversation at idigdata.com.